

UNISAL

Centro Universitário Salesiano de São Paulo

SISTEMA DE CONTROLE DE ESTOQUE DE PRODUTOS PERECÍVEIS

Documento Arquitetural Completo

Engenharia de Software

Integrantes

Ana Beatriz de Oliveira Elias

Elisa da Silva Ortiz de Godoy

Larissa dos Santos Oliveira

Vinicius Gama Bittencourt

Lorena - SP

2026

1 Evolução do Diagrama de Classes

1.1 Diagrama de Classes – Versão Original

A versão original do diagrama de classes foi construída diretamente a partir dos requisitos funcionais levantados na Parte 1 do projeto. Ela representa a estrutura inicial do Sistema de Controle de Estoque de Produtos Perecíveis, contemplando as entidades principais sem a aplicação formal de padrões de projeto.

Classes identificadas na versão original

- **Produto:** representa um item cadastrado no estoque, contendo atributos como `id`, `nome`, `categoria`, `unidadeMedida` e `estoqueMinimo`.
- **Lote:** representa uma remessa de produto com data de validade, quantidade, data de entrada e vínculo obrigatório com um Produto e um Fornecedor.
- **Fornecedor:** entidade responsável pelo abastecimento, contendo CNPJ, nome, contato e *leadTimeDias*.
- **Estoque:** centralizava operações de entrada, saída, consulta e alertas, tornando-se uma classe com múltiplas responsabilidades, caracterizando violação do Princípio da Responsabilidade Única (SRP).
- **Venda:** registrava saídas de produtos, armazenando data, quantidade e lote utilizado.
- **Relatorio:** responsável pela geração de relatórios de perdas e rastreamento de lotes (*recall*).
- **Etiqueta:** responsável pela impressão de etiquetas contendo QR Code e código de barras.

Problemas identificados na versão original

- A classe **Estoque** acumulava responsabilidades relacionadas a alertas, controle de vendas, geração de etiquetas e sugestão FEFO, violando o Princípio da Responsabilidade Única (SRP).
- Ausência de mecanismos de criação padronizada para objetos como Produto, Lote e Relatório, sendo as instâncias criadas diretamente com `new` no código cliente.
- A classe **GerenciadorEstoque** podia ser instanciada múltiplas vezes, causando possíveis inconsistências de estado.
- Não existia abstração para diferentes algoritmos de reposição, como FEFO, FIFO e estoque mínimo/máximo.
- Não havia separação clara entre a interface de acesso ao sistema e sua lógica interna, dificultando integrações externas.

- A geração de relatórios apresentava duplicação de estrutura entre suas diferentes variações, reduzindo a reutilização de código.

1.2 Diagrama de Classes – Versão Evoluída

A versão evoluída incorpora cinco padrões de projeto do catálogo Gang of Four (GoF), com o objetivo de corrigir os problemas identificados e aumentar a manutenibilidade, extensibilidade e coesão do sistema.

Classes e interfaces adicionadas

- **FabricaProduto** (interface), **FabricaProdutoPercivel** e **FabricaProdutoSeco**: responsáveis pela criação das instâncias de Produto conforme o tipo, aplicando o padrão **Factory Method**.
- **GerenciadorEstoque**: implementado como **Singleton**, garantindo uma única instância responsável pelo gerenciamento do estoque.
- **FachadaEstoque**: fornece um ponto único de entrada para as operações do sistema, encapsulando os subsistemas internos por meio do padrão **Facade**.
- **EstrategiaReposicao** (interface), juntamente com **ReposicaoFEFO**, **Reposicao-FIFO** e **ReposicaoEstoqueMinMax**: implementam algoritmos intercambiáveis de reposição por meio do padrão **Strategy**.
- **RelatorioTemplate** (classe abstrata), **RelatorioPerdasConcreta** e **Relatorio-RastreamentoConcreto**: estabelecem uma estrutura fixa para geração de relatórios utilizando o padrão **Template Method**.

1.3 Principais Alterações Estruturais

A Tabela 1.3 apresenta as principais diferenças entre a versão original e a versão evoluída do sistema após a aplicação dos padrões de projeto.

Aspecto	Versão Original	Versão Evoluída
Criação de objetos	new direto no código cliente	Factory Method com famílias de fábricas por tipo de produto
Instância do gerenciador	Múltiplas instâncias possíveis de GerenciadorEstoque	Singleton garante instância única e consistente
Acesso ao sistema	Módulos externos acessam subsistemas diretamente	Facade centraliza e simplifica a interface de uso
Algoritmo de reposição	Lógica fixa e acoplada na classe Estoque	Interface EstrategiaReposicao com implementações intercambiáveis
Geração de relatórios	Código duplicado em variantes de Relatorio	Template Method define esqueleto fixo com passos customizáveis

2 Documento Arquitetural – Aplicação dos Padrões GoF

Os cinco padrões aplicados foram escolhidos de forma que nenhum coincidissem com os padrões apresentados por outros grupos durante os seminários. Foram selecionados os padrões Factory Method, Singleton, Facade, Strategy e Template Method, todos aplicados ao domínio do Sistema de Controle de Estoque de Produtos Perecíveis.

2.1 Padrão 1: Factory Method [Criacional]

Problema Identificado

O sistema precisava criar diferentes tipos de Produto (perceíveis, secos e refrigerados) e Lote com configurações distintas. Na versão original, os objetos eram instanciados diretamente com **new** no código cliente, espalhando a lógica de criação por todo o sistema. Isso tornava impossível variar o tipo de objeto criado sem modificar cada ponto de instância, violando o Princípio Aberto/Fechado (OCP).

Solução Proposta

Aplicação do padrão Factory Method. A interface **FabricaProduto** declara o método **criarProduto()** como fábrica abstrata. As classes concretas **FabricaProdutoPerceivel** e **FabricaProdutoSeco** implementam a criação com regras específicas para cada família de produtos. O código cliente opera sobre a interface da fábrica sem conhecer a classe concreta instanciada.

Justificativa Técnica

O Factory Method delega a decisão de qual classe concreta instanciar para as subclasses e implementações, desacoplando o código cliente da lógica de construção. No contexto de produtos perecíveis, em que as regras de validação diferem por categoria, isso é essencial para a manutenibilidade do sistema.

Por que usar o padrão?

O sistema gerencia múltiplas categorias de produto com atributos e validações distintas. O Factory Method centraliza as regras de construção, elimina duplicação de código e permite a adição de novos tipos de produto sem alterar o código cliente.

Impacto Arquitetural

A lógica de criação foi encapsulada nas fábricas concretas. O `GerenciadorEstoque` e os serviços de importação passam a depender apenas da interface `FabricaProduto`, tornando a arquitetura mais extensível.

Vantagens

- Centraliza e encapsula as regras de construção;
- Permite adicionar novos tipos de produto sem modificar código existente;
- Facilita testes unitários;
- Elimina duplicação de código de validação.

Possíveis Limitações

- Introduz novas classes no sistema;
- Pode ser excessivo para sistemas simples;
- Requer disciplina para que os clientes dependam da interface da fábrica.

2.2 Padrão 2: Singleton [Criacional]

Problema Identificado

A classe `GerenciadorEstoque` é o núcleo do sistema, responsável pelo controle do inventário. Na versão original, era possível instanciar múltiplos objetos dessa classe, gerando inconsistências de estado entre diferentes partes da aplicação.

Solução Proposta

Aplicação do padrão Singleton. O construtor de `GerenciadorEstoque` é declarado privado e a instância é acessada por meio do método estático `getInstancia()`, garantindo que apenas uma instância seja utilizada em toda a aplicação.

Justificativa Técnica

Em sistemas de estoque em tempo real, múltiplas operações podem ocorrer simultaneamente. A existência de uma única instância centralizada garante que todas as operações atuem sobre o mesmo estado do inventário.

Por que usar o padrão?

O `GerenciadorEstoque` representa um recurso compartilhado e crítico para o negócio. O Singleton garante acesso controlado e uma única fonte de verdade para o estado do sistema.

Impacto Arquitetural

Todas as partes do sistema passam a acessar o estoque por meio de `GerenciadorEstoque.getInstancia()`, eliminando inconsistências e centralizando o gerenciamento do inventário.

Vantagens

- Garante consistência do estado do inventário;
- Fornece acesso global controlado;
- Permite inicialização sob demanda;
- Pode ser implementado de forma segura para ambientes concorrentes.

Possíveis Limitações

- Pode dificultar testes unitários;
- Introduz acoplamento implícito;
- Em arquiteturas distribuídas, a unicidade é limitada à instância da aplicação.

2.3 Padrão 3: Facade [Estrutural]

Problema Identificado

O sistema é composto por múltiplos subsistemas interdependentes, como `GerenciadorEstoque`, `ServicoAlerta`, `ServicoRelatorio`, `ServicoEtiqueta` e `ServicoReposicao`. Os módulos externos precisavam conhecer e interagir com cada subsistema individualmente, gerando

elevado acoplamento e tornando qualquer alteração interna impactante para os clientes externos.

Solução Proposta

Aplicação do padrão Facade. A classe **FachadaEstoque** oferece uma interface simplificada e unificada para o sistema, delegando internamente as operações aos subsistemas adequados.

Justificativa Técnica

O padrão Facade reduz a complexidade de integração ao expor apenas os casos de uso relevantes, ocultando os detalhes de implementação e permitindo a evolução dos subsistemas sem afetar os clientes externos.

Por que usar o padrão?

A operação de entrada de lotes envolve diversas etapas e subsistemas. A fachada encapsula essa orquestração em métodos únicos, simplificando a utilização do sistema.

Impacto Arquitetural

A classe **FachadaEstoque** torna-se o principal ponto de comunicação entre os clientes externos e o núcleo do sistema, reduzindo o acoplamento e facilitando a manutenção.

Vantagens

- Reduz o acoplamento entre clientes e subsistemas;
- Simplifica a utilização do sistema;
- Centraliza a orquestração dos serviços;
- Facilita a evolução da API.

Possíveis Limitações

- Pode concentrar responsabilidades excessivas;
- Clientes muito especializados podem precisar acessar os subsistemas diretamente;
- Não elimina a necessidade de testes individuais nos subsistemas.

2.4 Padrão 4: Strategy [Comportamental]

Problema Identificado

O sistema precisa suportar diferentes algoritmos de reposição e saída de estoque, como FEFO, FIFO e estoque mínimo/máximo. Na versão original, essa lógica estava concentrada em estruturas condicionais na classe `Estoque`, dificultando a manutenção e a adição de novos algoritmos.

Solução Proposta

Foi aplicada a interface `EstrategiaReposicao`, juntamente com as implementações `ReposicaoFEFO`, `ReposicaoFIFO` e `ReposicaoEstoqueMinMax`, encapsulando cada algoritmo em uma classe independente.

Justificativa Técnica

O padrão Strategy elimina condicionais complexas e permite variar o comportamento por meio da composição, respeitando os princípios OCP e DIP.

Por que usar o padrão?

Produtos perecíveis exigem critérios específicos de saída, enquanto outros produtos podem utilizar estratégias diferentes. O padrão permite adaptar o algoritmo conforme a categoria do produto.

Impacto Arquitetural

Os algoritmos foram desacoplados do `GerenciadorEstoque`, possibilitando a troca dinâmica da estratégia utilizada.

Vantagens

- Elimina condicionais complexas;
- Facilita testes unitários;
- Permite alterar algoritmos em tempo de execução;
- Favorece a conformidade com regras de negócio específicas.

Possíveis Limitações

- Requer conhecimento das estratégias disponíveis;
- Pode aumentar a quantidade de classes;
- A troca dinâmica exige cuidados em ambientes concorrentes.

2.5 Padrão 5: Template Method [Comportamental]

Problema Identificado

Os diferentes tipos de relatório compartilhavam a mesma sequência de execução, mas apresentavam duplicação de código e risco de inconsistência estrutural.

Solução Proposta

Foi criada a classe abstrata `RelatorioTemplate`, responsável por definir o fluxo fixo de geração dos relatórios. As subclasses implementam apenas as etapas específicas de cada tipo de relatório.

Justificativa Técnica

O Template Method garante que a sequência de passos seja preservada, permitindo variações apenas nas partes específicas do processo.

Por que usar o padrão?

Os relatórios precisam seguir uma estrutura padronizada, especialmente em situações relacionadas à conformidade regulatória e rastreamento sanitário.

Impacto Arquitetural

A duplicação de código foi eliminada e novos tipos de relatório podem ser adicionados apenas por meio da criação de novas subclasses.

Vantagens

- Elimina duplicação de código;
- Garante uma sequência padronizada de execução;
- Facilita a criação de novos relatórios;
- • Facilita conformidade regulatória: estrutura padronizada garantida pela classe base.

Possíveis Limitações

- • Acoplamento via herança: subclasses dependem da estrutura da classe abstrata, dificultando refatorações na hierarquia;
- P • Para relatórios com fluxos muito distintos, o Template Method pode forçar abstração artificial;

- Um grande número de métodos abstratos pode tornar as subclasses mais complexas.